

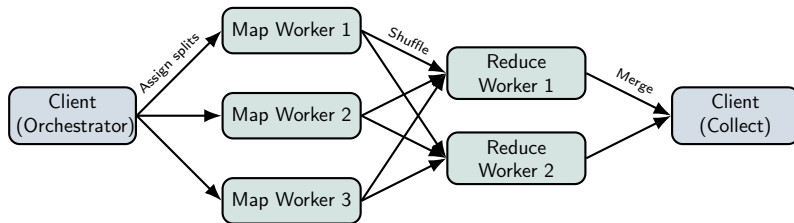
Distributed MapReduce on Lab Machines

Project presentation

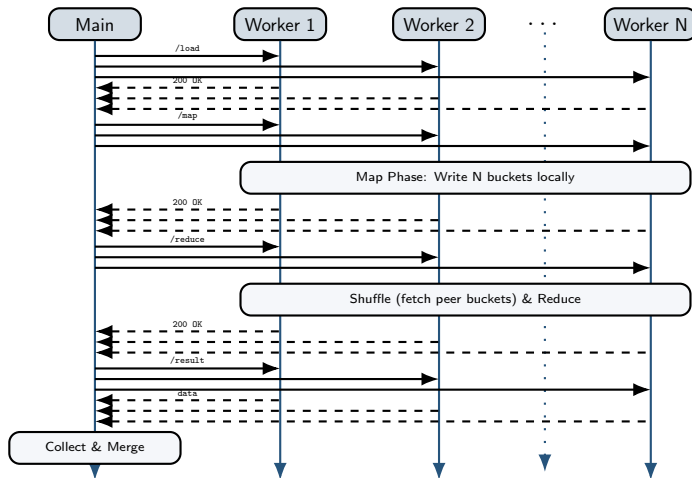
What We Built

- Automatic host discovery from `https://tp.telecom-paris.fr/ajax.php`
- Go orchestrator that builds, deploys, starts, monitors, and cleans workers
- HTTP worker API for `load -> map -> shuffle -> reduce -> result`
- Direct Common Crawl WET download from `data.commoncrawl.org`
- Four jobs: `wordcount`, `langdetect`, `domainpop`, `docdensity`

Architecture



Execution Sequence



- ➊ **Load:** Send ≤ 64 MB chunks or WET URLs.
- ➋ **Map:** Write N bucket files locally.
- ➌ **Shuffle+reduce:** Fetch peer buckets via HTTP.
- ➍ **Collect:** Merge & write result locally.

Map assignment: Orchestrator split + POST /data

Reducer assignment: $\text{FNV-32a}(\text{key}) \% N$

Failure Detection:

- A background health watcher polls `GET /health` every 5 seconds.
- Two consecutive failures mark a host as **dead**.

Recovery Execution (What happens when a node fails):

- The orchestrator pulls a **spare host** from the `hosts.txt` pool.
- It deploys a new worker binary over SSH to the spare.
- The system **replays** the `/load` and `/map` phases for the failed slot.
- If no spare host is available, the failed slot is reassigned to an existing healthy worker.
- Logical **slots** ($0..N-1$) stay fixed even if the physical host changes.

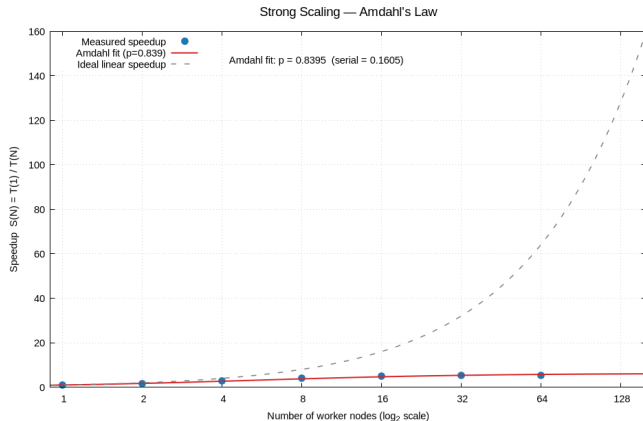
Use Cases Beyond Wordcount

Job	What it measures	Example output
wordcount	Token frequency	the 239298
langdetect	Language signal counts	lang:en, lang:fr
domainpop	Crawled domain popularity	doi.org, youtube.com
doccdensity	Content size and density	stat:lines, hist:len.*

Executing a specific job

```
# The orchestrator injects the selected job into the worker binary  
./mapreduce.sh -job scripts/jobs/langdetect \  
-commoncrawl -crawl CC-MAIN-2026-21 -n 8
```

Measurements and Amdahl's Law



Go Source Files (64 files):

- 1 node: 0.973 s (1.00x)
- 2 nodes: 0.494 s (1.97x)
- 4 nodes: 0.383 s (2.54x)
- 8 nodes: **0.266 s (3.66x)**
- 16 nodes: 1.412 s (0.69x)
- 64 nodes: 10.452 s (0.09x)

Methodology: Strong-scaling test on 64 Go files, varying active nodes from 1 to 64. Reference baseline is $N = 1$.

Batch MapReduce vs Kafka Streams

Custom MapReduce	Kafka Streams
Batch job with explicit phases	Continuous stream processing model
Lightweight HTTP + local temp files	Persistent Kafka log and state stores
Fast on small workloads (0.26s for 64 files on 8 nodes)	Slower on small batches (10.09s for 64 files)
Deployment struggled at 50 nodes due to SSH limits	Extremely robust at scale (completed 50 nodes in 248.5s)

Pain Points: What Broke and Why?

During development and testing at scale, we encountered major blockers:

- ❶ **Zombie Processes:** Killed orchestrators left orphan worker processes running on remote lab machines, locking up assigned ports.
 - *Fix:* Made cleanup idempotent (matching port/process ID) and added health monitoring.

Performance Optimization & Profiling

- **In-Mapper Combiner:** Introduce an aggregation tree or combiner step in the mapper to drastically reduce HTTP requests and intermediate data size.
- **Streaming IO:** Stream intermediate bucket results instead of keeping maps fully in memory. This prevents OOM crashes observed with large Common Crawl files at $N=1$ and $N=2$.

Fault Tolerance Improvements

- **Durable State:** The current MapReduce state is volatile in `/tmp`. Implementing replicated bucket storage could provide stronger, Hadoop-like fault tolerance without full phase replays.

Live demo path

```
(cd scripts && go test ./...)
./mapreduce.sh -job scripts/jobs/wordcount \
  -input test_input.txt -n 4 -output result.txt
./run_all_commoncrawl_jobs.sh -crawl CC-MAIN-2026-21 \
  -files-limit 1 -n 4
```